

Operators, Expressions এবং JavaScript-এ Data নিয়ে কাজ করা

Chapter 2-এ আমরা Data Types নিয়ে বিস্তারিত জেনেছি। কিন্তু শুধু data জমা রাখলে program চলে না। Program চলার জন্য data নিয়ে হিসাব করতে হয়, তুলনা করতে হয় এবং সিদ্ধান্ত নিতে হয়। এই সব কাজের জন্য প্রয়োজন **Operators এবং Expressions**।

PART 01 — OPERATORS, OPERANDS & EXPRESSIONS (1 – 4)

1 What is an Operator?

English: An **operator** is a symbol or keyword that tells JavaScript to perform an operation.

বাংলা: Operator হলো এমন একটি symbol বা keyword যা JavaScript-কে কোনো নির্দিষ্ট operation করতে বলে।

Example:

JAVASCRIPT

EXAMPLE

```
10 + 5
```

এখানে:

```
10 → Operand + → Operator 5 → Operand
```

Result:

```
15
```

সহজভাবে

```
Operand + Operator + Operand
```

```
10      +      5
      ↓
      15
```

2 What is an Operand?

Operator যে value-এর উপর কাজ করে, তাকে **operand** বলে।

Example:

JAVASCRIPT

EXAMPLE

```
20 - 8
```

এখানে:

20 → Operand - → Operator 8 → Operand

আর:

JAVASCRIPT

EXAMPLE

```
x * 10
```

এখানে:

x → Operand * → Operator 10 → Operand

3 What is an Expression?

English: An **expression** is a piece of code that produces a value.

বাংলা: Expression হলো এমন code যা একটি value produce করে।

Example:

JAVASCRIPT

EXAMPLE

```
10 + 20
```

Result:

```
30
```

আর:

JAVASCRIPT

EXAMPLE

```
age ≥ 18
```

Result হতে পারে:

```
true
```

আরও EXAMPLE

JAVASCRIPT

EXAMPLE

```
5 * 10
```

JAVASCRIPT

EXAMPLE

```
"Hello" + " World"
```

JAVASCRIPT

EXAMPLE

```
20 > 10
```

সবগুলো expression।

4 Types of Operators

JavaScript-এ অনেক ধরনের operator রয়েছে।

আমরা এই chapter-এ শিখব:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Equality Operators
5. Logical Operators
6. Unary Operators
7. Increment/Decrement
8. Ternary Operator
9. Nullish Coalescing Operator
10. Optional Chaining
11. Bitwise Operators
12. `typeof`
13. `in`
14. `instanceof`
15. Operator Precedence

PART 02 — ARITHMETIC OPERATORS & MODULUS (5 – 15)

5 Arithmetic Operators

Arithmetic operators mathematical calculation করার জন্য ব্যবহার হয়।

প্রধান arithmetic operators:

+ Addition - Subtraction * Multiplication / Division % Modulus ** Exponentiation

6 Addition +

JAVASCRIPT

EXAMPLE

```
const a = 10;
const b = 20;

console.log(a + b);
```

Output:

30

REAL-LIFE EXAMPLE

ধরো:

Product = ₹500 Delivery = ₹100

JAVASCRIPT

EXAMPLE

```
const productPrice = 500;
const deliveryCharge = 100;

const total = productPrice + deliveryCharge;

console.log(total);
```

Output:

600

7 String Concatenation with +

+ শুধু number যোগ করে না। String combine-ও করতে পারে।

JAVASCRIPT

EXAMPLE

```
const firstName = "Shariar";
const lastName = "Ripon";

console.log(firstName + " " + lastName);
```

Output:

Shariar Ripon

8 Subtraction -

JAVASCRIPT

EXAMPLE

```
const a = 50;  
const b = 20;  
  
console.log(a - b);
```

Output:

30

REAL-LIFE

JAVASCRIPT

EXAMPLE

```
const balance = 1000;  
const expense = 250;  
  
const remaining = balance - expense;  
  
console.log(remaining);
```

Output:

750

9 Multiplication *

JAVASCRIPT

EXAMPLE

```
const price = 500;  
const quantity = 3;  
  
console.log(price * quantity);
```

Output:

1500

REAL-LIFE

একটি product-এর price ৳500 এবং quantity 3 হলে total:

$500 \times 3 = 1500$

10 Division /

JAVASCRIPT

EXAMPLE

```
const totalMarks = 400;  
const subjects = 4;  
  
console.log(totalMarks / subjects);
```

Output:

100

11 Modulus %

এটা খুব important।

% remainder বা ভাগশেষ বের করে।

JAVASCRIPT

EXAMPLE

```
console.log(10 % 3);
```

Output:

```
1
```

কারণ:

```
10 ÷ 3 3 × 3 = 9 10 - 9 = 1
```

তাই:

```
10 % 3 = 1
```

12 Modulus-এর Real-Life Use

একটি number even না odd সেটা বের করতে % ব্যবহার করা যায়।

JAVASCRIPT

EXAMPLE

```
const number = 10;  
  
console.log(number % 2);
```

Output:

```
0
```

কারণ 10 সম্পূর্ণভাবে 2 দিয়ে divisible।

আর:

JAVASCRIPT

EXAMPLE

```
const number = 7;  
  
console.log(number % 2);
```

Output:

```
1
```

13 Even / Odd Concept

JAVASCRIPT

EXAMPLE

```
const number = 15;

if (number % 2 === 0) {
  console.log("Even");
} else {
  console.log("Odd");
}
```

Output:

Odd

এখানে আমরা ব্যবহার করেছি:

- `%`
- `===`
- `if`
- `else`

`if/else` আমরা পরের chapter-এ বিস্তারিত শিখব।

14 Exponentiation ******

Power বা exponent বের করার জন্য:

JAVASCRIPT

EXAMPLE

```
console.log(2 ** 3);
```

Output:

8

কারণ:

$2 \times 2 \times 2 = 8$

আর:

JAVASCRIPT

EXAMPLE

```
console.log(5 ** 2);
```

Output:

25

15 Arithmetic Operators Summary

Operator	কাজ	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Remainder	10 % 3	1
**	Power	2 ** 3	8

PART 03 — ASSIGNMENT & COMPOUND ASSIGNMENT (16 – 24)

16 Assignment Operator =

= হলো assignment operator।

এটি comparison করে না।

JAVASCRIPT

EXAMPLE

```
let age = 22;
```

এর অর্থ:

age variable-এর মধ্যে 22 value assign করো।

17 Reassignment

JAVASCRIPT

EXAMPLE

```
let score = 50;

score = 80;

console.log(score);
```

Output:

80

প্রথমে:

score → 50

পরে:

score → 80

18 Compound Assignment Operators

JavaScript-এ shortcut assignment operator আছে।

```
+= -= *= /= %= **=
```

19 +=

JAVASCRIPT

EXAMPLE

```
let score = 50;

score += 10;

console.log(score);
```

Output:

```
60
```

এটি essentially:

JAVASCRIPT

EXAMPLE

```
score = score + 10;
```

এর shortcut।

20 -=

JAVASCRIPT

EXAMPLE

```
let balance = 1000;

balance -= 250;

console.log(balance);
```

Output:

```
750
```

Equivalent:

JAVASCRIPT

EXAMPLE

```
balance = balance - 250;
```

21 

JAVASCRIPT

EXAMPLE

```
let price = 100;  
  
price *= 5;  
  
console.log(price);
```

Output:

500

Equivalent:

JAVASCRIPT

EXAMPLE

```
price = price * 5;
```

22 

JAVASCRIPT

EXAMPLE

```
let amount = 1000;  
  
amount /= 4;  
  
console.log(amount);
```

Output:

250

23 

JAVASCRIPT

EXAMPLE

```
let number = 17;  
  
number %= 5;  
  
console.log(number);
```

Output:

2

কারণ:

17 % 5 = 2

24 ****=**

JAVASCRIPT

EXAMPLE

```
let number = 2;  
  
number **= 3;  
  
console.log(number);
```

Output:

8

Equivalent:

JAVASCRIPT

EXAMPLE

```
number = number ** 3;
```

PART 04 – COMPARISON & EQUALITY OPERATORS (25 – 35)**25** **Comparison Operators**

Comparison operator দুইটি value compare করে এবং সাধারণত Boolean result দেয়:

true false

প্রধান operators:

> < ≥ ≤ = === ≠ !==

26 **Greater Than** >

JAVASCRIPT

EXAMPLE

```
console.log(10 > 5);
```

Output:

true

কারণ 10 বড় 5 থেকে।

27 **Less Than** <

JAVASCRIPT

EXAMPLE

```
console.log(5 < 10);
```

Output:

true

28 Greater Than or Equal >

JAVASCRIPT

EXAMPLE

```
console.log(10 >= 10);
```

Output:

```
true
```

কারণ 10, 10-এর সমান।

আর:

JAVASCRIPT

EXAMPLE

```
console.log(15 >= 10);
```

Output:

```
true
```

29 Less Than or Equal <=

JAVASCRIPT

EXAMPLE

```
console.log(10 <= 10);
```

Output:

```
true
```

30 Loose Equality =

= value compare করার সময় type coercion করতে পারে।

JAVASCRIPT

EXAMPLE

```
console.log(5 == "5");
```

Output:

```
true
```

কারণ JavaScript এখানে type conversion করে comparison করতে পারে।

31 Strict Equality

 value এবং type—দুটোই compare করে।

JAVASCRIPT

EXAMPLE

```
console.log(5 === "5");
```

Output:

false

কারণ:

5 → number "5" → string

Value দেখতে একই হলেও type আলাদা।

32 vs

এটা খুব ভালোভাবে মনে রাখবো।

JAVASCRIPT

EXAMPLE

```
console.log(10 == "10");
```

Output:

true

কিন্তু:

JAVASCRIPT

EXAMPLE

```
console.log(10 === "10");
```

Output:

false

BEGINNER/MODERN JS RULE ★

সাধারণত comparison করার সময়:

JAVASCRIPT

EXAMPLE



এবং:

JAVASCRIPT

EXAMPLE



ব্যবহার করা বেশি predictable।

33 Loose Inequality $\neq$

$\neq$ value equal নয় কিনা check করে এবং type coercion করতে পারে।

JAVASCRIPT

EXAMPLE

```
console.log(5  $\neq$  "5");
```

Output:

```
false
```

কারণ loose comparison অনুযায়ী তারা equal।

34 Strict Inequality $\neq$

$\neq$ value অথবা type—যেকোনো একটি আলাদা হলে `true` দেয়।

JAVASCRIPT

EXAMPLE

```
console.log(5  $\neq$  "5");
```

Output:

```
true
```

কারণ type আলাদা।

35 Comparison Summary

Operator	Meaning
$>$	Greater than
$<$	Less than
$\geq$	Greater than or equal
$\leq$	Less than or equal
$=$	Loose equality
$\equiv$	Strict equality
$\neq$	Loose inequality
$\neq$	Strict inequality

PART 05 — LOGICAL & SHORT-CIRCUIT OPERATORS (36 – 54)

36 Logical Operators

Logical operators একাধিক condition/value নিয়ে কাজ করতে পারে।

প্রধান তিনটি:

```
&& AND || OR ! NOT
```

37 AND &&

&& তখন true হয় যখন দুই পাশের condition-ই truthy/true হয়।

JAVASCRIPT

EXAMPLE

```
console.log(true && true);
```

Output:

true

কিন্তু:

JAVASCRIPT

EXAMPLE

```
console.log(true && false);
```

Output:

false

TRUTH TABLE

A	B	`A && B`
true	true	true
true	false	false
false	true	false
false	false	false

38 Real-Life AND Example

ধরো কোনো website-এ checkout করার জন্য:

User logged in AND Cart is not empty

দুটোই true হতে হবে।

JAVASCRIPT

EXAMPLE

```
const isLoggedIn = true;  
const hasItems = true;  
  
console.log(isLoggedIn && hasItems);
```

Output:

true

39 OR ||

|| -এর ক্ষেত্রে অন্তত একটি operand truthy হলে result truthy হতে পারে।

JAVASCRIPT

EXAMPLE

```
console.log(true || false);
```

Output:

```
true
```

TRUTH TABLE

A	B	A	B
true	true	true	
true	false	true	
false	true	true	
false	false	false	

40 Real-Life OR Example

ধরো login করা যাবে:

```
Google account OR Email/password
```

যেকোনো একটি থাকলেই চলবে।

JAVASCRIPT

EXAMPLE

```
const hasGoogleLogin = false;  
const hasEmailLogin = true;  
  
console.log(hasGoogleLogin || hasEmailLogin);
```

Output:

```
true
```

41 NOT !

! Boolean value উল্টে দেয়।

JAVASCRIPT

EXAMPLE

```
console.log(!true);
```

Output:

```
false
```

আর:

JAVASCRIPT

EXAMPLE

```
console.log(!false);
```

Output:

```
true
```

42 Real-Life NOT Example

JAVASCRIPT

EXAMPLE

```
const isLoggedIn = false;  
  
console.log(!isLoggedIn);
```

Output:

```
true
```

কারণ:

```
isLoggedIn = false !false = true
```

43 Double NOT !!

!! কোনো value-কে Boolean-এ convert করার একটি common technique।

JAVASCRIPT

EXAMPLE

```
console.log(!! "Hello");
```

Output:

```
true
```

আর:

JAVASCRIPT

EXAMPLE

```
console.log(!! 0);
```

Output:

```
false
```

কারণ প্রথম ! এবং দ্বিতীয় ! মিলিয়ে Boolean conversion-এর মতো behavior দেয়।

44 Logical Operators শুধু Boolean Return করে?

এখানে JavaScript-এর একটি খুব important behavior আছে।

`&&` এবং `||` সবসময় `true` বা `false` return করে না।

তারা operand-এর actual value return করতে পারে।

Example:

JAVASCRIPT

EXAMPLE

```
console.log("Hello" && "World");
```

Output:

World

কারণ `"Hello"` truthy, তাই `&&` পরের operand evaluate করে এবং `"World"` return করে।

45 `&&` Short-Circuit

JAVASCRIPT

EXAMPLE

```
console.log(false && "Hello");
```

Output:

false

কারণ প্রথম value-ই falsy।

JavaScript দ্বিতীয় operand-এর দিকে যেতে হয় না।

এটাকে বলে:

Short-circuit evaluation

46 `||` Short-Circuit

JAVASCRIPT

EXAMPLE

```
console.log("Hello" || "World");
```

Output:

Hello

কারণ `"Hello"` truthy।

JavaScript প্রথম truthy value পেয়ে থেমে গেছে।

47 Practical Default Value Pattern

পুরোনো/common pattern:

JAVASCRIPT

EXAMPLE

```
const username = userInput || "Guest";  
  
console.log(username);
```

যদি `userInput` falsy হয়, `"Guest"` ব্যবহার হবে।

তবে মনে রাখবে— `||` সব falsy value-কে fallback হিসেবে treat করে।

এজন্য modern JavaScript-এ অনেক ক্ষেত্রে `??` বেশি precise।

48 Nullish Coalescing ??

`??` হলো modern JavaScript-এর খুব useful operator।

এটি fallback দেয় যখন left side:

`null`

অথবা:

`undefined`

হয়।

Example:

JAVASCRIPT

EXAMPLE

```
const username = null;  
  
console.log(username ?? "Guest");
```

Output:

`Guest`

49 ?? vs ||

এটা খুব important!

JAVASCRIPT

EXAMPLE

```
console.log(0 || 100);
```

Output:

100

কারণ 0 falsy।

কিন্তু:

JAVASCRIPT

EXAMPLE

```
console.log(0 ?? 100);
```

Output:

0

কারণ 0 null বা undefined নয়।

তাই:

|| → সব falsy value-এর ক্ষেত্রে fallback হতে পারে ?? → শুধু null বা undefined-এর ক্ষেত্রে fallback

50 Practical Example of ??

ধরো user's age 0 valid value হতে পারে।

JAVASCRIPT

EXAMPLE

```
const age = 0;  
  
console.log(age ?? 18);
```

Output:

0

কিন্তু:

JAVASCRIPT

EXAMPLE

```
console.log(age || 18);
```

Output:

18

তাই context অনুযায়ী operator নির্বাচন করতে হবে।

51 Optional Chaining ?.

ধরো একটি nested object:

JAVASCRIPT

EXAMPLE

```
const user = {  
  profile: {  
    name: "Ripon"  
  }  
};
```

তুমি লিখতে পারো:

JAVASCRIPT

EXAMPLE

```
console.log(user.profile.name);
```

Output:

Ripon

কিন্তু যদি `profile` না থাকে?

JAVASCRIPT

EXAMPLE

```
const user = {};  
  
console.log(user.profile.name);
```

এতে error হবে।

52 Optional Chaining দিয়ে

JAVASCRIPT

EXAMPLE

```
const user = {};  
  
console.log(user.profile?.name);
```

Output:

undefined

?. বলে:

যদি এই property/object null বা undefined হয়, তাহলে error না দিয়ে undefined return করো।

53 Optional Chaining Example

JAVASCRIPT

EXAMPLE

```
const user = {
  profile: {
    address: {
      city: "Dhaka"
    }
  }
};

console.log(user.profile?.address?.city);
```

Output:

Dhaka

আর:

JAVASCRIPT

EXAMPLE

```
const user = {};

console.log(user.profile?.address?.city);
```

Output:

undefined

54 Optional Chaining with Functions

`?.()` দিয়ে optional function call করা যায়।

JAVASCRIPT

EXAMPLE

```
const user = {
  greet() {
    console.log("Hello!");
  }
};

user.greet?.();
```

Output:

Hello!

যদি function না থাকে:

JAVASCRIPT

EXAMPLE

```
const user = {};

user.greet?.();
```

তাহলে error না দিয়ে call skip করতে পারে।

55 Increment Operator ++

Variable-এর value 1 বাড়াতে:

JAVASCRIPT

EXAMPLE

```
let count = 5;

count++;

console.log(count);
```

Output:

6

Equivalent:

JAVASCRIPT

EXAMPLE

```
count = count + 1;
```

56 Decrement Operator --

Value 1 কমাতে:

JAVASCRIPT

EXAMPLE

```
let count = 5;

count--;

console.log(count);
```

Output:

4

Equivalent:

JAVASCRIPT

EXAMPLE

```
count = count - 1;
```

57 Prefix vs Postfix

এটা একটু tricky।

POSTFIX

JAVASCRIPT

EXAMPLE

```
let x = 5;  
  
console.log(x++);
```

Output:

5

কিন্তু পরে:

JAVASCRIPT

EXAMPLE

```
console.log(x);
```

Output:

6

PREFIX

JAVASCRIPT

EXAMPLE

```
let x = 5;  
  
console.log(++x);
```

Output:

6

DIFFERENCE

$x++$ → আগে old value use, পরে increment $++x$ → আগে increment, পরে new value use

58 Prefix/ Postfix Example

JAVASCRIPT

EXAMPLE

```
let a = 10;

let b = a++;

console.log(a);
console.log(b);
```

Output:

```
11 10
```

কারণ `a++` প্রথমে `10` return করেছে, তারপর `a` হয়েছে `11`।

এখন:

JAVASCRIPT

EXAMPLE

```
let a = 10;

let b = ++a;

console.log(a);
console.log(b);
```

Output:

```
11 11
```

কারণ `++a` আগে increment করেছে।

59 Unary Operators

যে operator একটি operand-এর উপর কাজ করে তাকে unary operator বলা হয়।

Examples:

JAVASCRIPT

EXAMPLE

```
typeof value
```

JAVASCRIPT

EXAMPLE

```
!value
```

JAVASCRIPT

EXAMPLE

```
++value
```

JAVASCRIPT

EXAMPLE

```
--value
```

এখানে এক operand নিয়ে operation হচ্ছে।

60 Unary Plus +

String number-কে number-এ convert করার জন্য unary + ব্যবহার করা যায়।

JAVASCRIPT

EXAMPLE

```
const value = "100";  
  
console.log(+value);
```

Output:

100

Type:

JAVASCRIPT

EXAMPLE

```
console.log(typeof +value);
```

Output:

number

তবে beginner হিসেবে explicit:

JAVASCRIPT

EXAMPLE

```
Number(value)
```

অনেক সময় বেশি readable।

61 Unary Minus -

JAVASCRIPT

EXAMPLE

```
const value = "10";  
  
console.log(-value);
```

Output:

-10

এখানে string "10" numeric conversion-এর মধ্য দিয়ে গেছে।

62 Ternary Operator ? :

Ternary operator হলো ছোট `if/else` condition লেখার concise way।

Syntax:

JAVASCRIPT

EXAMPLE

```
condition ? valueIfTrue : valueIfFalse
```

Example:

JAVASCRIPT

EXAMPLE

```
const age = 20;

const result = age ≥ 18 ? "Adult" : "Minor";

console.log(result);
```

Output:

Adult

63 Ternary Operator Breakdown

এই code:

JAVASCRIPT

EXAMPLE

```
const result = age ≥ 18 ? "Adult" : "Minor";
```

মানে:

```
যদি age ≥ 18
  ↓
  Adult
নাহলে
  ↓
  Minor
```

64 Real-Life Ternary Example

JAVASCRIPT

EXAMPLE

```
const isLoggedIn = true;

const message = isLoggedIn
  ? "Welcome back!"
  : "Please login";

console.log(message);
```

Output:

Welcome back!

কখন ব্যবহার করবে?

Simple condition-এর জন্য।

খুব complex condition হলে normal `if/else` বেশি readable।

65 Bitwise Operators

এগুলো comparatively advanced।

Bitwise operators number-এর binary representation-এর bit নিয়ে কাজ করে।

প্রধান operators:

```
& | ^ ~ << >> >>>
```

এগুলো web development-এর daily beginner work-এ খুব বেশি লাগে না, কিন্তু JavaScript language-এর অংশ হিসেবে জানা উচিত।

66 Bitwise AND &

JAVASCRIPT

EXAMPLE

```
console.log(5 & 3);
```

Binary:

```
5 → 101 3 → 011
```

AND:

```
101 011 --- 001
```

```
001 = 1
```

Output:

```
1
```

67 Bitwise OR |

JAVASCRIPT

EXAMPLE

```
console.log(5 | 3);
```

Binary:

```
101 011 --- 111
```

```
111 = 7
```

Output:

```
7
```

68 Bitwise XOR ^

JAVASCRIPT

EXAMPLE

```
console.log(5 ^ 3);
```

Binary:

```
101 011 --- 110
```

110 = 6

Output:

```
6
```

69 Bitwise NOT ~

JAVASCRIPT

EXAMPLE

```
console.log(~5);
```

Output:

```
-6
```

এটি two's complement representation-এর কারণে হয়।

Bitwise concepts low-level programming, flags, certain algorithms ইত্যাদিতে useful হতে পারে।

70 Left Shift <<

JAVASCRIPT

EXAMPLE

```
console.log(5 << 1);
```

Output:

```
10
```

Binary:

```
5 = 101 101 << 1 = 1010 = 10
```

71 Right Shift >>

JAVASCRIPT

EXAMPLE

```
console.log(10 >> 1);
```

Output:

```
5
```

72 Zero-fill Right Shift >>>

JAVASCRIPT

EXAMPLE

```
console.log(10 >>> 1);
```

Output:

5

>> এবং >>> negative number-এর ক্ষেত্রে গুরুত্বপূর্ণভাবে আলাদা behave করতে পারে।

73 typeof Operator

Chapter 2-এ আমরা এটা দেখেছি।

JAVASCRIPT

EXAMPLE

```
const name = "Ripon";  
  
console.log(typeof name);
```

Output:

string

আর:

JAVASCRIPT

EXAMPLE

```
console.log(typeof 100);
```

Output:

number

74 in Operator

`in` দিয়ে object-এর মধ্যে কোনো property আছে কিনা check করা যায়।

JAVASCRIPT

EXAMPLE

```
const user = {  
  name: "Ripon",  
  age: 22  
};  
  
console.log("name" in user);
```

Output:

```
true
```

আর:

JAVASCRIPT

EXAMPLE

```
console.log("email" in user);
```

Output:

```
false
```

75 instanceof

`instanceof` দিয়ে কোনো object কোনো constructor/class-এর instance কিনা check করা যায়।

JAVASCRIPT

EXAMPLE

```
const fruits = [];  
  
console.log(fruits instanceof Array);
```

Output:

```
true
```

আর:

JAVASCRIPT

EXAMPLE

```
const date = new Date();  
  
console.log(date instanceof Date);
```

Output:

```
true
```

OOP এবং prototypes শেখার সময় `instanceof` আরও পরিষ্কার হবে।

76 Operator Precedence

একটি expression-এ একাধিক operator থাকলে কোনটা আগে execute হবে?

এটা নির্ধারণ করে **operator precedence**।

Example:

JAVASCRIPT

EXAMPLE

```
console.log(10 + 5 * 2);
```

অনেকে ভাবতে পারে:

$$10 + 5 = 15 \quad 15 \times 2 = 30$$

কিন্তু output:

20

কারণ multiplication আগে:

$$5 \times 2 = 10 \quad 10 + 10 = 20$$
77 Parentheses () দিয়ে Priority Control

যদি তুমি addition আগে করতে চাও:

JAVASCRIPT

EXAMPLE

```
console.log((10 + 5) * 2);
```

Output:

30

কারণ:

$$10 + 5 = 15 \quad 15 \times 2 = 30$$
GOLDEN RULE ★

যদি precedence নিয়ে confusion হয়, parentheses ব্যবহার করো।

78 Common Precedence Idea

সাধারণভাবে:

```
( )  
↓  
**  
↓  
*, /, %  
↓  
+, -  
↓  
>, <, ≥, ≤  
↓  
=, ==, ≠, !==  
↓  
&&  
↓  
||  
↓  
??  
↓  
?:  
↓  
assignment
```

এটা পুরো specification-এর সম্পূর্ণ table নয়; বরং practical mental model।

79 A Big Example

এখন অনেক operator একসাথে:

JAVASCRIPT

EXAMPLE

```
const price = 500;  
const quantity = 3;  
const discount = 100;  
  
const total = price * quantity - discount;  
  
console.log(total);
```

Calculation:

```
500 × 3 = 1500 1500 - 100 = 1400
```

Output:

```
1400
```

80 Real-World Shopping Example

JAVASCRIPT

EXAMPLE

```
const productPrice = 1200;
const quantity = 2;
const discount = 300;
const deliveryCharge = 100;

const subtotal = productPrice * quantity;
const total = subtotal - discount + deliveryCharge;

console.log("Subtotal:", subtotal);
console.log("Total:", total);
```

Output:

```
Subtotal: 2400 Total: 2200
```

81 Logical + Comparison Example

JAVASCRIPT

EXAMPLE

```
const age = 22;
const hasId = true;

const canEnter = age ≥ 18 && hasId;

console.log(canEnter);
```

Output:

```
true
```

এখানে:

```
age ≥ 18
↓
true

hasId
↓
true

true && true
↓
true
```

82 || + ?? Example

JAVASCRIPT

EXAMPLE

```
const username = "";

console.log(username || "Guest");
```

Output:

Guest

কারণ empty string falsy।

কিন্তু:

JAVASCRIPT

EXAMPLE

```
const username = "";

console.log(username ?? "Guest");
```

Output:

এখানে empty string `null` বা `undefined` নয়, তাই fallback হয়নি।

83 Optional Chaining + Nullish Coalescing

এই combination real-world frontend development-এ অনেক useful।

JAVASCRIPT

EXAMPLE

```
const user = {};

const city = user.profile?.address?.city ?? "Unknown";

console.log(city);
```

Output:

Unknown

এখানে:

```
user.profile
  ↓
undefined
  ↓
?. prevents error
  ↓
undefined
  ↓
?? "Unknown"
  ↓
"Unknown"
```

84 Common Mistake — = vs ===

❌ ভুল ধারণা:

JAVASCRIPT

EXAMPLE

```
if (age = 18)
```

এখানে = assignment operator।

Comparison করতে সাধারণত:

JAVASCRIPT

EXAMPLE

```
if (age === 18)
```

ব্যবহার করবে।

85 Common Mistake — = vs ===

JAVASCRIPT

EXAMPLE

```
console.log(0 = false);
```

Output:

```
true
```

কিন্তু:

JAVASCRIPT

EXAMPLE

```
console.log(0 === false);
```

Output:

```
false
```

কারণ:

```
0 → number false → boolean
```

Modern JS-এ strict equality সাধারণত safer/predictable choice।

86 Common Mistake — String + Number

JAVASCRIPT

EXAMPLE

```
console.log("Price: " + 100 + 50);
```

Output:

```
Price: 10050
```

কারণ প্রথম `+`-এর পর String concatenation শুরু হয়েছে।

যদি $100 + 50$ করতে চাও:

JAVASCRIPT

EXAMPLE

```
console.log("Price: " + (100 + 50));
```

Output:

```
Price: 150
```

87 Common Mistake — Floating Point

JavaScript-এর floating-point arithmetic-এ কিছু surprising result হতে পারে।

JAVASCRIPT

EXAMPLE

```
console.log(0.1 + 0.2);
```

Output:

```
0.30000000000000004
```

কেন?

কারণ computer সাধারণত decimal fraction-কে binary floating-point representation-এ store করে, এবং কিছু decimal value exactভাবে represent করা যায় না।

এটা JavaScript-এর একার সমস্যা নয়; IEEE 754 floating-point systems-এর সাধারণ limitation।

88 Financial Calculation-এর ক্ষেত্রে সতর্কতা

যেমন:

JAVASCRIPT

EXAMPLE

```
console.log(0.1 + 0.2);
```

Expected:

```
0.3
```

কিন্তু actual:

```
0.30000000000000004
```

Real-world financial systems-এ floating-point-এর বদলে integer smallest unit (যেমন cents/paisa) বা appropriate decimal/money handling ব্যবহার করা হয়।

এটা advanced application development-এ খুব important।

89 Chapter 3 — Operator Cheat Sheet**ARITHMETIC**`+ - * / % **`**ASSIGNMENT**`= += -= *= /= **=`**COMPARISON**`> < ≥ ≤`**EQUALITY**`= == ≠ ≡`**LOGICAL**`&& || !`**INCREMENT/DECREMENT**`++ --`**CONDITIONAL**`? :`**MODERN OPERATORS**`?? ?.`**OTHER**`typeof in instanceof`**BITWISE**`& | ^ ~ << >> >>>`

🧠 Chapter 3 — Quick Revision

এক লাইনে মনে রাখো:

মৌলিক সারমর্ম: Operator data-এর উপর operation করে, আর expression সেই operation-এর মাধ্যমে একটি value produce করতে পারে।

EXPRESSION STRUCTURE

10 + 20

10 + 20

এখানে:

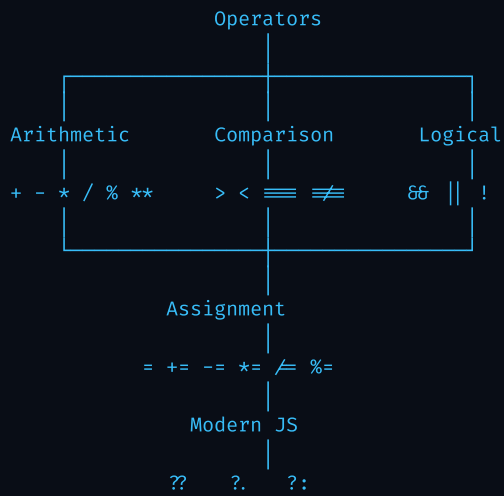
10 → Operand

+ → Operator

20 → Operand

30 → Result

Operators-এর বড় পিকচার (Mental Architecture):



Chapter 3 — Practice Set

এগুলো আগে **নিজে output predict করবে**, তারপর browser console-এ run করবে:

Practice 1 (Basic Arithmetic & Modulus):

```
console.log(20 + 10);
console.log(20 - 10);
console.log(20 * 10);
console.log(20 / 10);
console.log(20 % 3);
console.log(2 ** 5);
```

Practice 2 (Compound Assignment Flow):

```
let score = 100;

score += 20;
score -= 10;
score *= 2;
score /= 2;

console.log(score);
```

Final output কী হবে?

Practice 3 (Comparison & Equality Contrast):

```
console.log(10 > 5);
console.log(10 < 5);
console.log(10 ≥ 10);
console.log(10 ≤ 9);
console.log(10 === "10");
console.log(10 == "10");
```

প্রতিটির output explain করো।

Practice 4 (Logical Truth Tables):

```
console.log(true && false);
console.log(true || false);
console.log(!true);
console.log(!false);
```

Practice 5 (String vs Number Arithmetic Coercion):

```
console.log("5" + 2);
console.log("5" - 2);
console.log("5" * 2);
console.log("5" / 2);
```

কেন প্রথমটির behavior অন্যগুলোর থেকে আলাদা—নিজে explain করার চেষ্টা করো।

Practice 6 (Ternary Operator):

```
const age = 17;  
const result = age ≥ 18 ? "Adult" : "Minor";  
console.log(result);
```

Practice 7 (Optional Chaining `?.`):

```
const user = {  
  profile: {  
    name: "Ripon"  
  }  
};  
  
console.log(user.profile?.name);  
console.log(user.address?.city);
```

Practice 8 — Real Project Logic:

ধরো:

```
Product Price = 1500  
Quantity = 3  
Discount = 500  
Delivery = 100
```

JavaScript দিয়ে **Subtotal** ও **Final Total** বের করো।

Chapter 3 Mini Project — Shopping Cart Calculator

নিজে এই logic তৈরি করার চেষ্টা করো:

Cart Calculation Structure:

Product Price

Quantity

Discount

Delivery Charge

Subtotal = (Price * Quantity)

Final Total = Subtotal - Discount + Delivery Charge

Example Data:

Product Price: 800 | Quantity: 2 | Discount: 100 | Delivery: 60

Expected: Subtotal: 1600 | Final Total: 1560

JAVASCRIPT

SHOPPING CART CALCULATOR IMPLEMENTATION

```
const productPrice = 800;
const quantity = 2;
const discount = 100;
const deliveryCharge = 60;

// 1. Subtotal calculation
const subtotal = productPrice * quantity;

// 2. Final Total calculation
const finalTotal = subtotal - discount + deliveryCharge;

console.log("Subtotal:", subtotal);           // 1600
console.log("Final Total:", finalTotal);       // 1560
```

এখানে ব্যবহৃত হয়েছে: Variables, Arithmetic operators, Assignment এবং `console.log()`

★ Chapter 3-এর Must-Know Concepts

```
+ - * / % **
= += -= *= /=
> < ≥ ≤
= == ≠ ===
&& || !
++ --
?:
??
?.
typeof
```

এর মধ্যে `===`, `&&`, `||`, `??`, `?.`, `%`, `++/--`, ternary এবং operator precedence বিশেষভাবে ভালোভাবে practice করবে।

✦ পরবর্তী অধ্যায় — Chapter 4 Preview

Chapter 4 — Control Flow & Decision Making

এখানে JavaScript কীভাবে **decision নেয়** সেটা শুরু হবে:

```
if, else, else if, nested if  
switch, case, default, break, fall-through  
truthy, falsy, logical conditions, ternary
```

এরপর আমরা ধাপে ধাপে **Loops** → **Functions** → **Strings** → **Arrays** → **Objects** → **Scope** → **DOM** → **Events** → **Async JavaScript** → **APIs** → **OOP** → **Advanced JavaScript** → **JavaScript Internals**-এর দিকে এগিয়ে যাব।